DAY 5 DSA

Complete Lecture-Based Study Notes

Linked Lists • Recursion • Stacks • Queues • Expression Conversion

Prepared from the Day 5 lecture transcript uploaded in this chat

Note: These notes follow the uploaded transcript's topics. Explanatory pseudocode, dry runs, examples and practice questions are added as study aids; they are not claimed to be the instructor's exact original code.

How to Study This Lecture

This lecture revisits a linked-list predicate problem, recursive reversal, stack-based expression evaluation and conversion, then introduces queue variations and applications. Focus on tracing the data structure after every operation rather than memorizing code.

Lecture map

• Counting linked-list elements that satisfy a Predicate / lambda condition.

• Finding the middle of a linked list and understanding recursive reversal.

• Postfix and prefix evaluation using a stack.

• Converting infix expressions to postfix and prefix.

• Queue applications and parenthesis balancing.

• Linear queues, circular queues and circular-queue implementation.

• Deque (double-ended queue) and priority queue.

> The transcript's closing remarks mention hashing and trees as topics planned for the following lecture. They are not treated as core Day 5 topics here because the uploaded lecture mainly covers the subjects listed above.

1. Counting Linked-List Elements with a Predicate

A Predicate is a condition that takes a value and returns true or false. In Java, **Predicate<T>** is a functional interface whose abstract method is **test(T value)**. It is useful when a method should count values matching a condition without hard-coding that condition inside the method.

Example: count even numbers

```
Predicate<Integer> isEven = n -> n % 2 == 0;

int count = 0;
Node<Integer> trav = head;

while (trav != null) {
    if (isEven.test(trav.data)) {
        count++;
    }
    trav = trav.next;
}
```

The expression **n -> n % 2 == 0** is a lambda expression. It receives n and returns a boolean. The call **isEven.test(trav.data)** evaluates the condition for the current node.

Reusable countIf method

```
int countIf(Predicate<Integer> condition) {
    int count = 0;
    Node<Integer> trav = head;

    while (trav != null) {
        if (condition.test(trav.data)) {
            count++;
        }
        trav = trav.next;
    }
    return count;
}

// Examples:
countIf(n -> n % 2 == 0);  // count even values
countIf(n -> n > 10);      // count values greater than 10
countIf(n -> n < 0);       // count negative values
```

Why use Predicate? The traversal/counting logic stays the same, while the condition can change each time the method is called.

> **Complexity:** O(n) time because each node is visited once; O(1) extra space for the iterative method.

Dry run

Node value	Condition: even?	count after visit
7	false	0
4	true	1
12	true	2
9	false	2

Final answer: 2 even values. Remember that a predicate tests one value; the loop applies it to every node.

2. Finding the Middle of a Linked List

The slow-and-fast pointer technique finds the middle in one traversal. The slow pointer moves one node per iteration, while the fast pointer moves two.

```
Node slow = head;
Node fast = head;

while (fast != null && fast.next != null) {
    slow = slow.next;
    fast = fast.next.next;
}
return slow;
```

Example with an odd number of nodes

For 10 → 20 → 30 → 40 → 50, slow progresses 10 → 20 → 30 while fast progresses 10 → 30 → 50. The loop then stops and slow points to 30.

Even number of nodes

For 10 → 20 → 30 → 40, the common version above returns 30, the second of the two middle nodes. To return the first middle, the initial pointers or loop condition need to change.

Complexity: O(n) time and O(1) auxiliary space. The method also avoids counting the full list first and then traversing again.

3. Recursive Reversal of a Linked List

A recursive method calls itself on a smaller subproblem. To reverse a singly linked list recursively, keep calling the function on head.next until the last node is reached.

```
Node reverse(Node head) {
    if (head == null || head.next == null) {
        return head;
    }

    Node newHead = reverse(head.next);
    head.next.next = head;
    head.next = null;
    return newHead;
}
```

Understand the two pointer statements

• **head.next.next = head;** makes the next node point back to the current node.

• **head.next = null;** removes the old forward link from the current node, which becomes the tail of the reversed portion.

Example: 10 → 20 → 30 → null. The recursive calls reach node 30. While returning, 20.next is made to point to 10, and 10.next is set to null. Node 30 remains the new head.

> **Complexity:** O(n) time and O(n) call-stack space. This differs from iterative reversal, which can use O(1) extra space.

Common errors

• Forgetting the base case, causing recursion not to stop.

• Returning the original head instead of newHead.

• Forgetting to set head.next = null, which can create a cycle.

• Assuming the original head remains the first node after reversal.

4. Stack Refresher

A stack follows **LIFO (Last In, First Out)**. Insertions and removals happen at the top.

Operation	Meaning	Typical time
push(x)	Add x at the top	O(1)
pop()	Remove and return top	O(1)
peek()/top()	Read top without removing	O(1)
isEmpty()	Check if empty	O(1)

Stack underflow happens when pop/peek is attempted on an empty stack. Overflow can happen in a fixed-capacity array stack when it is full.

Where stacks are used

• Function calls and recursion (call stack).

• Undo operations and browser history patterns.

• Parenthesis balancing.

• Infix-to-postfix/prefix conversion.

• Postfix and prefix expression evaluation.

5. Infix, Postfix and Prefix

Notation	Operator position	Example for (A + B) * C
Infix	Between operands	(A + B) * C
Postfix	After operands	A B + C *
Prefix	Before operands	* + A B C

Infix is the notation commonly used in mathematics. In postfix and prefix, operator placement encodes the order of operations, so evaluation can be done using a stack without repeatedly applying normal precedence rules.

Precedence and associativity

Operator	Usual precedence	Typical associativity
Parentheses ( )	Highest grouping	—
^ (if included)	High	Often right-to-left; language/convention dependent
*, /, %	Above + and -	Left-to-right
+ , -	Lower	Left-to-right

Always follow the operator rules specified by the question. Exponentiation associativity can differ across languages and teaching conventions, so do not assume it is left-associative.

6. Evaluating a Postfix Expression

Postfix evaluation scans left to right. Push operands. When an operator appears, pop the right operand first, then the left operand. Apply the operator as left operator right, and push the result.

Algorithm

```
for each token from left to right:
    if token is an operand:
        push(token)
    else:
        right = pop()
        left = pop()
        result = left token right
        push(result)

answer = pop()
```

Worked example: 5 2 3 * +

Token	Action	Stack (bottom → top)
5	Push 5	[5]
2	Push 2	[5, 2]
3	Push 3	[5, 2, 3]
*	2 × 3 = 6; push 6	[5, 6]
+	5 + 6 = 11; push 11	[11]

Result: 11. **Order warning:** for subtraction and division, do not calculate first-popped operand minus/divided by second-popped operand. The first popped item is the right operand.

Complexity: O(n) time and O(n) worst-case stack space for n tokens.

7. Evaluating a Prefix Expression

Prefix evaluation scans from right to left. Push operands. When an operator appears, pop the first operand and then the second, apply the operator in the intended order, and push the result.

Example: * + 5 2 3

This represents (5 + 2) × 3 = 21. Scanning right to left, push 3, push 2, push 5; the + operation combines 5 and 2 to make 7; then * combines 7 and 3 to make 21.

Prefix and postfix use the same basic stack idea, but their scan direction and operand pop order differ. Trace the stack carefully, especially for - and /.

8. Converting Infix to Postfix

The conversion algorithm uses an output sequence and a stack for operators. Operands go directly to output; operators wait on the stack until precedence/parentheses determine when they should be output.

Rules

• If the token is an operand, append it to output.

• If the token is an opening parenthesis (, push it.

• If the token is a closing parenthesis), pop operators to output until the matching (is found; discard the parentheses.

• For an operator, pop operators with higher precedence from the stack. For equal precedence, use associativity rules; for common left-associative operators, pop equal-precedence operators too.

• At the end, pop all remaining operators to output.

Worked example: A + B * C

Token	Output	Operator stack
A	A	[]
+	A	[+]
B	A B	[+]
*	A B	[+, *]
C	A B C	[+, *]
End	A B C * +	[]

The multiplication operator has higher precedence than addition, so postfix becomes **A B C * +**, which means A + (B × C).

Parentheses example: (A + B) * C

The opening parenthesis prevents + from being output too early. On reading), output + and discard the matching (. The final postfix form is **A B + C ***.

9. Converting Infix to Prefix

A common stack-based method is to reverse the infix expression, swap opening and closing parentheses, convert the modified expression to postfix, then reverse the resulting postfix sequence to get prefix. However, equal-precedence associativity handling must be adjusted carefully when reversing; follow the algorithm your course uses.

Example

Infix: (A + B) * C

Prefix: * + A B C

The expression tree idea is useful: the root operator is *, its left expression is + A B, and its right operand is C.

> Do not blindly reverse characters in multi-digit operands. Treat operands/tokens as units, and be careful with associativity for operators such as subtraction, division and exponentiation.

10. Parenthesis Balancing

A stack can check whether opening and closing brackets are properly matched and nested.

Algorithm

```
for each character ch:
    if ch is '(' or '[' or '{':
        push(ch)
    else if ch is ')' or ']' or '}':
        if stack is empty:
            return false
        open = pop()
        if open does not match ch:
            return false

return stack is empty
```

Examples

Input	Balanced?	Reason
(a+b)	Yes	One matching pair.
((a+b)*c)	Yes	Pairs are properly nested.
(a+b))	No	An extra closing bracket.
([)]	No	Types close in the wrong order.
((a+b)	No	An opening bracket remains.

Complexity: O(n) time and O(n) space in the worst case. The algorithm checks both matching bracket type and nesting order.

11. Queue Fundamentals and Applications

A queue follows **FIFO (First In, First Out)**. New elements are inserted at the rear; removal happens at the front.

Operation	Meaning	Typical time
enqueue(x)	Insert at rear	O(1) with suitable implementation
dequeue()	Remove from front	O(1) with suitable implementation
front()/peek()	Read front item	O(1)
isEmpty()	Check if empty	O(1)

Example

enqueue(10), enqueue(20), enqueue(30) gives front-to-rear order [10, 20, 30]. dequeue() returns 10, leaving [20, 30].

Queue applications

• Scheduling jobs/tasks in arrival order.

• Breadth-first search (BFS) in graphs and trees.

• Handling requests in buffers or service systems.

• Producer-consumer pipelines and asynchronous processing.

An implementation that shifts every remaining array element after dequeue can take O(n) per removal. A circular array or linked-list implementation can support O(1) enqueue/dequeue under normal assumptions.

12. Linear Queue vs Circular Queue

In a simple linear array queue, rear can reach the last array index even if earlier positions became free after dequeue. This can waste available array space unless elements are shifted.

A circular queue treats the array as if its last position connects back to index zero. The next index is computed with modulo arithmetic.

```
nextIndex = (index + 1) % capacity
```

Circular queue state

For capacity 5, the index after 4 is (4 + 1) % 5 = 0. Therefore the rear can wrap around to the beginning if that position is free.

Common array implementation

```
enqueue(x):
    if count == capacity:
        report overflow
    rear = (rear + 1) % capacity
    array[rear] = x
    count++

dequeue():
    if count == 0:
        report underflow
    x = array[front]
    front = (front + 1) % capacity
    count--
    return x
```

Using a separate count makes full and empty conditions clear: count == capacity means full; count == 0 means empty. Other implementations use front/rear sentinel conventions, so check the chosen design before applying the conditions.

Complexity

Enqueue and dequeue are O(1); the queue uses O(capacity) storage. The modulo operation wraps the index without shifting items.

13. Double-Ended Queue (Deque)

A deque (pronounced 'deck') supports insertion and removal at both ends. It generalizes a regular queue and a stack.

Operation	Meaning
addFirst(x)	Insert at the front
addLast(x)	Insert at the rear
removeFirst()	Remove from the front
removeLast()	Remove from the rear
peekFirst()/peekLast()	Inspect either end

A deque can behave like a queue by inserting at the rear and removing at the front. It can behave like a stack by inserting and removing at the same end.

Common applications

• Sliding-window maximum/minimum algorithms.

• Palindrome checking from both ends.

• Implementing stack and queue behaviour in one structure.

• Work-stealing and scheduling strategies in some systems.

A deque is not the same thing as a priority queue: a deque’s operations are based on its two ends, while a priority queue removes the item with the highest or lowest priority.

14. Priority Queue

A priority queue removes elements according to priority rather than simply insertion order. For example, an emergency task may be handled before a routine task even if the routine task arrived first.

A priority queue can be implemented with an unsorted array/list, a sorted array/list, or a heap. These choices have different costs.

Implementation	Insert	Remove highest/lowest priority
Unsorted list/array	O(1) append	O(n) search for best priority

Implementation	Insert	Remove highest/lowest priority
Sorted list/array	O(n) insertion	O(1) remove at chosen end
Binary heap	O(log n)	O(log n)

A binary heap is a common implementation. A min-priority queue returns the smallest key first; a max-priority queue returns the largest key first. The meaning of 'highest priority' depends on the convention.

Do not confuse these structures

• **Queue:** order is based on arrival (FIFO).

• **Priority queue:** order is based on priority.

• **Deque:** operations can happen at both ends.

15. Complexity Summary

Problem / operation	Time	Extra space
Predicate count over linked list	O(n)	O(1)
Middle node with slow/fast pointers	O(n)	O(1)
Recursive linked-list reversal	O(n)	O(n) call stack
Postfix/prefix evaluation	O(n)	O(n) worst case
Parenthesis balancing	O(n)	O(n) worst case
Circular queue enqueue/dequeue	O(1)	O(capacity) storage
Deque end operations (good implementation)	O(1)	Depends on implementation
Priority queue with binary heap	O(log n) insert/remove	O(n) storage

16. Interview Questions and Quick Answers

Q: What does Predicate.test() return?
A: A boolean: true when the supplied value satisfies the condition, otherwise false.

Q: Why is a predicate useful in countIf?
A: It separates the traversal logic from the condition being checked.

Q: How do slow and fast pointers find the middle?
A: Slow moves one step and fast two; slow reaches the middle when fast reaches the end.

Q: Why does recursive reversal return newHead?
A: The former last node becomes the new head and must be returned through recursive calls.

Q: Which direction is used for postfix evaluation?
A: Left to right.

Q: Which direction is used for prefix evaluation?
A: Right to left.

Q: In postfix evaluation, which operand is popped first?
A: The right operand; the next popped operand is the left operand.

Q: How does a stack check balanced parentheses?
A: Push opening brackets; on closing brackets, pop and verify the type; finish with an empty stack.

Q: Why use a circular queue?
A: It reuses freed array positions without shifting elements.

Q: What is a deque?
A: A double-ended queue that supports insertion/removal at both ends.

Q: How does a priority queue differ from a normal queue?
A: Removal depends on priority, not just arrival order.

17. Practice Questions

1. For list 3 → 8 → 10 → 12, count the even values using a predicate.
2. What does countIf(n -> n > 5) return for 2 → 6 → 9 → 1?
3. Find the middle node of 1 → 2 → 3 → 4 → 5 using slow/fast pointers.
4. Reverse 4 → 7 → 9 recursively. What does the returned list look like?
5. Evaluate postfix: 6 2 / 3 +.
6. Evaluate prefix: - 9 / 8 2.
7. Convert A + B * C to postfix.
8. Convert (A + B) * C to prefix.
9. Is ([{}]) balanced? Is ([)] balanced? Explain.
10. A circular queue has capacity 6 and rear=5. What is the next rear index after enqueue, assuming space is available?
11. Give one real-world application each for a queue, deque and priority queue.
12. Compare linear queue and circular queue in terms of wasted array positions.

Answer key

1. 3 → false, 8 → true, 10 → true, 12 → true; count = 3.
2. 2 values (6 and 9).
3. Node 3.
4. 9 → 7 → 4 → null.
5. 6 / 2 = 3, then 3 + 3 = 6.
6. 8 / 2 = 4, then 9 - 4 = 5.
7. A B C * +.
8. * + A B C.
9. ([{}]) is balanced; ([)] is not because the bracket types close in the wrong order.
10. (5 + 1) % 6 = 0.
11. Queue: BFS/task scheduling; deque: sliding-window algorithms; priority queue: processing highest-priority jobs first.
12. A linear array queue may leave freed positions unused after rear reaches the end; a circular queue wraps indices and reuses them.

18. Final Revision Sheet

Topic	Remember
Predicate	test(value) returns boolean; lambda provides condition
Middle node	slow +1, fast +2
Recursive reverse	base case null/one node; reverse links while returning
Postfix	scan left → right; pop right then left
Prefix	scan right → left; maintain correct operand order
Infix → postfix	operands to output; operators on stack; precedence + parentheses
Balanced brackets	stack of openings; each closing must match the latest opening
Queue	FIFO; insert rear, remove front
Circular queue	next index = (index + 1) % capacity
Deque	insert/remove at both ends
Priority queue	remove based on priority, not arrival time

Best practice order: (1) implement countIf with Predicate, (2) trace recursive reversal, (3) solve postfix/prefix evaluation by hand, (4) convert infix expressions, and (5) implement a circular queue with full/empty checks.